

Helixor QWMS Hardening and Re-Centering Execution Plan

Helixor QWMS Hardening and Re-Centering Execution Plan

Date: 2026-06-16

Repository: HorizonCore-Solutions-Ltd/Logivox

Current Branch: main

Plan Type: Founder/CTO Execution Roadmap

Execution Model: No rewrites, no schema reset, no big-bang migration

Executive Directive

Turn LogiVox into a clean, hardened, enterprise-ready QWMS platform by recomposition and hardening, not rewrite.

North Star

Build and package a voice-native, cognitive QWMS suite where:

1. QWMS Core is the center.
2. Voice is the workflow execution engine.
3. Cognitive layer orchestrates decisions and actions.
4. CAPA/QMS and other domains remain as modular enterprise add-ons.

0. Strategic Positioning (Why)

The platform already includes all major enterprise layers:

1. WMS core operations.
2. Voice execution engine.
3. Cognitive decision engine.
4. CAPA/QMS.
5. Workforce and duties.
6. Yard/Dock/Gate.
7. Replenishment intelligence.
8. Integrations and webhooks.
9. Reporting and analytics.

Strategic decision:

1. Keep all domains.
2. Re-center around QWMS + Voice.
3. Repackage non-core as modules.

Non-goals:

1. No rewrite of core runtime.
2. No model teardown/rebuild.
3. No CAPA removal.

Program Structure

1. Phase 1: Stabilize Contracts and Reliability.
2. Phase 2: Harden Tenant and Security Controls.
3. Phase 3: Align Domain Maturity and Runtime Behavior.
4. Phase 4: Institutionalize Maintainability and Operability.
5. Phase 5: Re-center Product Around QWMS Core.
6. Phase 6: Add Transport Optimization Premium Module.
7. Phase 7: Elevate Voice to Execution Driver.
8. Phase 8: Package Final Enterprise Suite.

Program Governance

1. Cadence: weekly planning, daily delivery sync, weekly architecture review.
2. Tracking: one board with Epics by phase and strict Definition of Done (DoD).
3. Release policy: phased cutovers behind feature flags.
4. Rollback policy: required for every production-impacting change.

Phase 1 - API Contract and Reliability Stabilization (1-2 weeks)

Goal: remove the highest-frequency breakpoints and eliminate contract drift.

Scope

1. Voice API contract normalization.
2. Duplicate route/import cleanup.
3. Webhook model consistency.
4. Contract tests for high-fragility surfaces.

Work Packages

1. Voice contract normalization.
 - Add compatibility endpoints:
 - /api/voice/transcribe
 - /api/voice/command
 - Keep /api/voice as canonical execution route.
 - Add translation layer from compatibility endpoints to canonical handler.
 - Update mobile client to canonical request/response schema.
 - Add API contract tests for voice endpoints.
2. Duplicate route/import cleanup.
 - Clean repeated imports and duplicate declarations in:
 - gate entries routes
 - picking task routes
 - Refactor shared logic to helper functions where duplication exists.
3. Webhook model consistency.
 - Select canonical model usage pattern.
 - Refactor route handlers and services to one model contract.
 - Validate event delivery logging and replay behavior.
4. Contract test suite additions.
 - Voice contract tests.
 - Mobile offline sync contract tests.
 - Webhook delivery and failure-path tests.
 - Replenishment trigger contract tests.

Exit Criteria

1. Voice contracts pass web and mobile test matrix.
2. Duplicate blocks removed from targeted routes.
3. Webhook model mismatch resolved and tested.
4. CI contains contract suite for listed domains.

KPI Targets

1. Zero contract-drift regressions in voice/mobile paths.
2. 100% passing contract tests in CI.

Phase 2 - Tenant and Security Hardening (1-2 weeks)

Goal: enforce enterprise-safe multi-tenant and authorization behavior everywhere.

Scope

1. Universal tenant guards.
2. Authorization matrix per domain.
3. High-risk audit log guarantees.

Work Packages

1. Tenant enforcement pass.
 - Every list/mutate route must validate:
 - organizationId scope
 - user/session identity
 - role authorization
2. Domain authorization matrix.
 - Define and codify permissions for:
 - WMS
 - Voice
 - CAPA
 - Yard/Dock/Gate
 - Workforce
 - Replenishment
 - Integrations

3. Audit guarantees.

- Add or verify audit logging for:
 - stock changes
 - task creation and completion
 - CAPA updates
 - gate entries
 - webhook failures/retries

Exit Criteria

1. Tenant guard checklist passes for all domain routes.
2. Authorization matrix implemented and tested.
3. Audit logs present for all high-risk actions.

KPI Targets

1. Zero unauthorized cross-tenant access in test suite.
2. 100% high-risk mutation routes emit audit records.

Phase 3 - Domain Maturity Alignment (2-4 weeks)

Goal: ensure all modules behave like production modules, not mixed maturity.

Scope

1. Replace placeholder/mock/default behavior.
2. Standardize service pipeline patterns.
3. Promote shared orchestration primitives.

Work Packages

1. Placeholder elimination campaign.
 - Inventory endpoints and handlers with mock/default behavior.
 - Replace with real service-backed logic.
2. Domain service pattern standardization.
 - Enforce standard flow:
 - validation
 - auth/tenant check
 - idempotency where needed
 - event publishing where needed
 - consistent error envelope
3. Shared primitives promotion.
 - Consolidate under shared lib:
 - validation wrappers
 - auth guards
 - tenant scoping helpers
 - event publish/retry primitives
 - error serialization

Exit Criteria

1. Placeholder inventory is reduced to zero in production path.
2. Standard request pipeline used by all prioritized domains.
3. Shared primitives adopted by target modules.

KPI Targets

1. Reduction in domain-specific duplicated guard/validation code.
2. Reduced variance in API error envelope shape.

Phase 4 - Maintainability and Operability (ongoing)

Goal: prevent drift and preserve quality under rapid feature growth.

Scope

1. ADR program.
2. Documentation drift controls.
3. Domain ownership model.
4. Expanded quality gates.

Work Packages

1. ADRs.
 - Create ADRs for:
 - Voice architecture
 - CAPA boundaries
 - Replenishment strategy
 - Event/outbox model
 - Tenant security model
 - Cognitive orchestration
2. Documentation CI rules.
 - If domain code changes beyond threshold, require docs delta.
 - Fail PR if required docs are missing.
3. Ownership model.
 - Assign maintainers by module:
 - WMS
 - Voice
 - CAPA/QC
 - Replenishment
 - Yard/Dock/Gate
 - Workforce
 - Integrations
4. Test gate expansion.
 - Add/expand:
 - load tests
 - security tests
 - integration tests
 - end-to-end business flow suites

Exit Criteria

1. ADR baseline exists for all listed strategic domains.
2. Docs drift checks active in CI.
3. Module ownership published and active.
4. Expanded test suites in release gate.

Phase 5 - Re-Centering to QWMS Core (Strategic Product Layer)

Goal: establish clean product center and modular boundaries without rewrites.

Core Product Declaration

QWMS Core includes:

1. Inventory.
2. Tasks.
3. Waves.
4. Replenishment.
5. Slotting.
6. OMS and shipments.
7. Voice execution engine.
8. Cognitive orchestration engine.

Modular Product Surround

Treat as attachable modules:

1. CAPA/QMS.
2. Workforce.
3. Yard/Dock/Gate.
4. Integrations.
5. Reporting.
6. Sustainability.

API Contract Unification

1. Adopt canonical API contract definition (OpenAPI or typed contract framework).
2. Generate:
 - mobile clients
 - web clients
 - shared types
 - contract tests

Branding Transition

1. Platform rename path: LogiVox -> Helixor QWMS.
2. Introduce alias period for backward compatibility in docs and artifacts.

Exit Criteria

1. Published core vs module boundary map.
2. Canonical API contract artifact generated and versioned.
3. Product naming transition plan approved.

Phase 6 - Transport Optimization Module (New Expansion)

Goal: monetize natural adjacent capability from existing yard/dock/shipment assets.

Scope

1. Route planning.
2. Load consolidation.
3. Carrier selection optimization.
4. Delivery window optimization.
5. Driver execution app flows.
6. POD capture and confirmation.

Exit Criteria

1. Transport Optimization module defined and gated behind commercial flag.
2. End-to-end pilot flow passes acceptance tests.

Phase 7 - Voice as Execution Engine (Differentiation)

Goal: move voice from feature to primary execution mode.

Scope

Voice-first execution across:

1. Picking.
2. Putaway.
3. Replenishment.
4. Cycle count.
5. QC actions.
6. Gate entry.
7. Yard moves.

Cognitive Integration

1. Voice intent emits structured execution events.
2. Cognitive engine evaluates context and proposes action.
3. Task orchestration executes and logs outcomes.

Exit Criteria

1. Voice-first paths available for prioritized workflows.
2. Cognitive-assisted orchestration active for selected flows.

Phase 8 - Final Enterprise Packaging

Goal: package platform as Helixor Logistics Suite.

Package Components

1. QWMS Core.
2. Voice Engine.
3. Cognitive Engine.
4. Replenishment.
5. Transport Optimization.
6. CAPA/QMS.
7. Workforce.
8. Yard/Dock/Gate.
9. Integrations.
10. Reporting.

Exit Criteria

1. SKU/module packaging and pricing maps ready.
2. Implementation playbook and release checklist finalized.

Implementation Sequencing (When)

Sprint Sequence

1. Sprint 1-2
 - Complete Phase 1
2. Sprint 3-4
 - Complete Phase 2
3. Sprint 5-8
 - Complete Phase 3
4. Sprint 9 onward
 - Phase 4 becomes continuous governance lane
5. Strategic Track (parallel)
 - Start Phase 5 in sprint 5
 - Start Phase 6 after Phase 3 stabilization
 - Start Phase 7 with Phase 6 pilots
 - Execute Phase 8 after successful pilots and packaging decisions

RACI-Lite Ownership Model

1. CTO/Founder
 - final architecture decisions
 - module boundary approval
 - packaging strategy
2. Platform Lead
 - shared primitives
 - CI quality gates
 - security and tenant hardening
3. Domain Leads
 - WMS, Voice, CAPA/QC, Replenishment, Yard, Workforce, Integrations
 - own implementation and test quality in their module
4. QA/Release Lead
 - contract/integration/e2e gate enforcement
 - release readiness sign-off

Quality Gates (Mandatory)

1. Contract tests required for any API contract change.
2. Tenant/security tests required for all route-level auth changes.
3. Integration tests required for webhook/event pipeline changes.
4. E2E tests required for workflow-level behavior changes.
5. Docs update required for domain behavior changes.

Risk Register and Mitigations

1. Risk: contract drift between mobile and web.
 - Mitigation: canonical contracts + generated clients + contract tests.
2. Risk: hidden regressions in duplicated logic.
 - Mitigation: deduplicate + service extraction + targeted tests.
3. Risk: inconsistent tenant protection.
 - Mitigation: universal guard policy + route audit checklist.
4. Risk: module complexity perception.
 - Mitigation: product boundary map (core vs modules) and packaging clarity.
5. Risk: maintenance overhead at scale.
 - Mitigation: ADRs, ownership tags, CI drift controls, release gates.

CAPA Decision

Decision: Keep CAPA.

Justification:

1. CAPA is deeply integrated with quality operations and duties.
2. CAPA is an enterprise differentiator, not accidental complexity.
3. Removal would reduce product value and coherence.
4. Complexity should be modularized, not amputated.

Definition of Success

Success is achieved when:

1. Contracts are stable and generated across clients.
2. Security and tenant controls are universal and test-enforced.
3. Placeholder logic is eliminated from production paths.
4. Core and module boundaries are explicit and commercialized.
5. Voice and cognitive execution become measurable workflow drivers.

Immediate Next 10 Actions

1. Approve this plan as execution baseline.
2. Create epics per phase in tracker.
3. Assign module owners.
4. Start Phase 1 voice compatibility implementation.
5. Clean duplicate route/import logic in gate/picking APIs.
6. Standardize webhook model usage.
7. Add contract tests for voice/mobile/webhook/replenishment.
8. Build tenant-guard route audit checklist.
9. Define authorization matrix per domain.
10. Publish ADR template and first six ADRs.

Artifact Output

1. This execution plan markdown file.
2. Matching PDF export for distribution.